FACULTY OF ELECTRONIC ENGINEERING & TECHNOLOGY UNIVERSITI MALAYSIA PERLIS

NMJ11004 COMPUTER PROGRAMMING SEM1 2024/2025

CASHIER SYSTEM

MINI PROJECT FINAL REPORT

GROUP NAME	
1. ANAAS MOHAMMED ABBAS MAHDI	231020061-1
2. ASHRAF SAIF ALI OBAD	231020049-5
3. ZEYAD TAREQ MUBARAK ALKUWAIL	231020027-5
4. WALEED KHALED BIN HELABI	231020025-5

Table of Contents

1 Introduction

1.1 Introduction to the Project

This project presents an Inventory and Billing Management System developed in C, designed to streamline the management of users, products, and sales transactions for small businesses. The system is divided into three main modules: User Management, Product Management, and Billing System. The User Management module handles user authentication and access control, while the Product Management module allows for adding, updating, and tracking product details. The Billing System module facilitates the creation of sales bills and stores transaction data for future reference.

Built with a menu-driven interface, the system ensures ease of use and incorporates input validation and error handling for data integrity. Data is stored persistently using file handling, allowing information to be retained between sessions. This project demonstrates the application of key programming concepts such as file handling, data structures, and modular programming, providing an efficient and scalable solution for inventory and billing management.

The project was collaboratively developed by a team of students, with each member contributing specific functionalities:

Ashraf: Designed the system, implemented the main menu, input validation, and the "Show Users List" feature.
Annas: Developed the "Delete User," "Update User," and "Add User" functionalities.
Zeyad: Implemented the "Delete Product," "Update Product," and "Add Product" features.
Walled: Worked on the billing system, including "Add Sales Bill" and "Show Sales Bills."

1.2 Objective

- To create a secure system that restricts access to authorized users only, ensuring data protection and preventing unauthorized access.
- To streamline and automate retail operations, improving efficiency and reducing manual errors.
- To provide a user-friendly interface for managing products, users, and sales transactions.
- To save time by simplifying the process of adding, updating, and deleting products, as well as generating and viewing sale bills.
- To maintain data integrity and security by implementing role-based access control, ensuring only authorized personnel can perform specific tasks.
- To enhance the overall user experience by providing clear menus, input validation, and error handling.

1.3 Significant of the Project

Advantages of the Project:

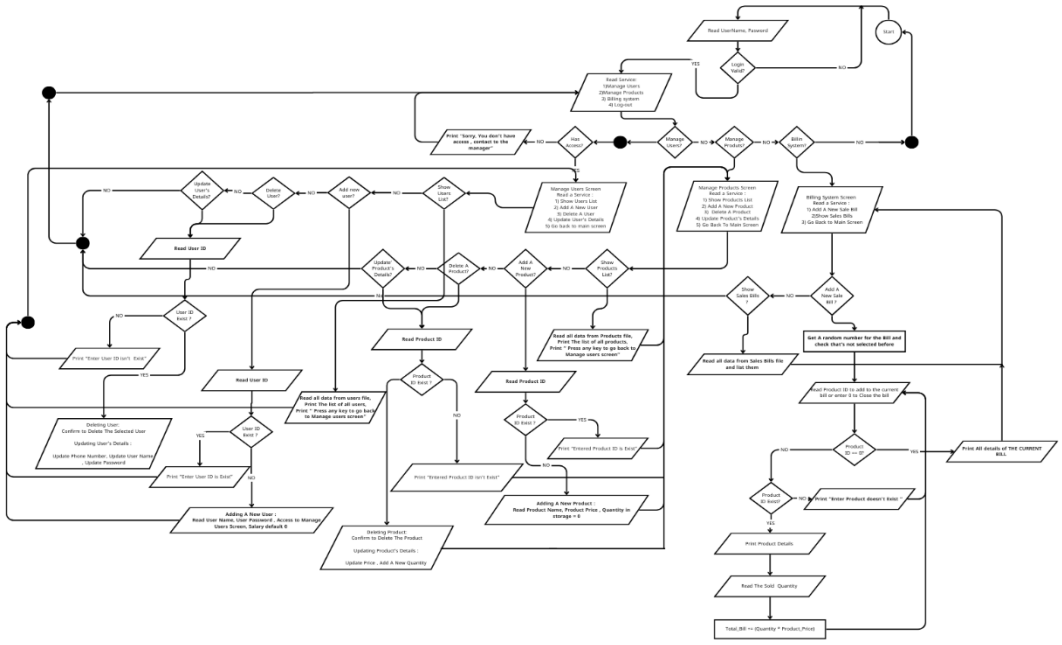
The proposed Point of Sale (POS) system offers numerous benefits to users, including improved efficiency and accuracy in managing products, sales, and user data. By automating tasks such as product management, billing, and inventory tracking, the system reduces manual effort and minimizes errors. It ensures data security through role-based access control, allowing only authorized users to perform specific functions. The user-friendly interface, with clear menus and input validation, enhances the overall experience, making it accessible even for non-technical users. Additionally, the system provides real-time updates and persistent data storage, ensuring that all information is up-to-date and retrievable even after system shutdown.

Scope of the Project:

The scope of the project includes user management, allowing administrators to add, update, delete, and view user details with role-based access control. It also covers product management, enabling users to add, update, delete, and view product information, including price and quantity. The billing system facilitates the generation and viewing of sale bills, complete with product details, quantities, and total amounts. Additionally, the system provides basic reporting features, allowing users to view sales transactions and product details for better decision-making. These functionalities are designed to streamline retail operations and improve overall efficiency.

2 Methodology

2.1 Flowchart



2.2 Design of the Project (input, process, output)

Table 1: System Design

List of System Functionalities	**Input, Process, Output, and type of data**	**C Techniques**
Login	Input: User ID (int), Password (string) Process: Validate credentials against stored user data. Output: Grant access to the main menu or display an error message.	The C Techniques used for the Login functionality include file handling (fopen, fscanf, fclose) to read stored user data, string comparison (strcmp) to validate passwords, and conditional statements (if-else) to check if credentials match. Additionally, input validation ensures the entered data is in the correct format. These techniques work together to securely authenticate users and handle invalid inputs gracefully.
Add User	Input: User ID (int), Name (string), Phone (string), Access Level (int), Password (string), Salary (float) Process: Append new user data to the users file. Output: Confirmation message that the user was added successfully.	The **C Techniques** used for **Add User** include **file handling** (fopen, fprintf, fclose) to append new user data to the file, **structs** to organize user details, and **input validation** to ensure data integrity. Additionally, **conditional statements** (if-else) check if the user ID already exists, preventing duplicates. These techniques ensure the system handles user inputs gracefully and stores data securely.

Delete User	Input: User ID (int) Process: Search for the user in the file and remove their record. Output: Confirmation message that the user was deleted successfully.	The C Techniques used for Delete User include file handling (fopen, fscanf, fclose) to read and rewrite the user file, conditional statements (if-else) to identify and skip the user to be deleted, and file manipulation (remove, rename) to update the file. These techniques ensure the system locates and removes the user record efficiently while maintaining data integrity.
Update User	Input: User ID (int), Updated fields (name, phone, password, access, salary) Process: Search for the user and update the specified fields in the file. Output: Confirmation message that the user was updated successfully.	The C Techniques used for Update User include file handling (fopen, fscanf, fprintf, fclose) to read and rewrite user data, conditional statements (if-else) to locate the user and update specific fields, and input validation to ensure updated data is valid. These techniques allow the system to modify user records efficiently while maintaining data accuracy and integrity.
Add Product	Input: Product ID (int), Name (string), Price (float), Quantity (int)	The **C Techniques** used for **Add Product** include **file handling** (fopen, fprintf, fclose) to append new product data to the file, **structs** to organize product

	Process: Append new product data to the products file. Output: Confirmation message that the product was added successfully.	details, and **input validation** to ensure data integrity. Additionally, **conditional statements** (if-else) check if the product ID already exists, preventing duplicates. These techniques ensure the system handles product inputs gracefully and stores data securely.
Delete Product	Input :Product ID (int) Process: Search for the product in the file and remove its record. Output :Confirmation message that the product was deleted successfully.	The **C Techniques** used for **Delete Product** include **file handling** (fopen, fscanf, fclose) to read and rewrite the product file, **conditional statements** (if-else) to identify and skip the product to be deleted, and **file manipulation** (remove, rename) to update the file. These techniques ensure the system locates and removes the product record efficiently while maintaining data integrity.
Update Product	Input: Product ID (int) Process: Search for the product in the file and remove its record. Output: Confirmation message that the product was deleted successfully.	The **C Techniques** used for **Update Product** include **file handling** (fopen, fscanf, fprintf, fclose) to read and rewrite product data, **conditional statements** (if-else) to locate the product and update specific fields, and **input validation** to ensure updated data is valid. These techniques allow the system to modify product records efficiently while maintaining data accuracy and integrity.
Add Sale Bill	Input: Product ID (int), Quantity (int)	The **C Techniques** used for **Add Sale Bill** include **file handling** (fopen, fprintf, fclose) to save bill details, **structs** to organize

	Process: Calculate the total price, generate a bill number, and save the bill to the sale bills file. Output: Confirmation message that the bill was added successfully, along with the bill details.	bill data, and **mathematical operations** to calculate the total price. Additionally, **conditional statements** (if-else) validate product availability and quantity, ensuring accurate billing. These techniques enable the system to generate and store bills efficiently while maintaining data integrity.
Show Sale Bill	Input: None Process: Read and display all sale bills from the sale bills file. Output: List of sale bills with details (bill number, date, user ID, product name, price).	The **C Techniques** used for **Show Sale Bill** include **file handling** (fopen, fscanf, fclose) to read bill data from the file, **loops** (for or while) to iterate through all records, and **formatting** (printf) to display bill details in a structured manner. These techniques ensure the system retrieves and presents sale bill information clearly and efficiently.

2.3 Function of the System

1. Login Functionality:

- **C Techniques:**
  - **File Handling (fopen, fscanf, fclose):** Reads user credentials from the file.
  - **String Comparison (strcmp):** Validates the entered password against the stored password.
  - **Conditional Statements (if-else):** Checks if the user ID and password match.

- **Explanation:**
  The login function ensures secure access to the system by validating user credentials. It reads user data from a file, compares the input password with the stored password, and grants access if they match.

Code Snapshot:

```
FILE *file = fopen("users.txt", "r");
if (!file) {
    printf("Error opening user file.\n");
    return;
}

User user;
bool valid = false;

while (fscanf(file, "%d#//#%[^#]#//#%[^#]#//#%d#//#%[^#]#//#%f\n",
              &user.id, user.name, user.phone, &user.access, user.password, &user.salary) != EOF) {
    if (id == user.id && strcmp(input_password, user.password) == 0) {
        valid = true;
        break;
    }
}

fclose(file);

if (valid) {
    printf("Login successful!\n");
} else {
    printf("Invalid username or password.\n");
}
```

2. Add Product Functionality:

- **C Techniques:**
  - **File Handling (fopen, fprintf, fclose):** Appends new product data to the file.
  - **Structs:** Organizes product details (ID, name, price, quantity).
  - **Input Validation:** Ensures the product ID is unique and data is valid.

- **Explanation:**
  This function allows users to add new products to the system. It ensures data integrity by checking for duplicate product IDs and stores the product details in a file.

Code Snapshot:

```
FILE *file = fopen("products.txt", "a");
if (!file) {
    printf("Error opening products file.\n");
    return;
}

Product product;
printf("Enter Product ID: ");
scanf("%d", &product.id);

// Check if product ID already exists
if (product_exists(product.id)) {
    printf("Product with ID %d already exists.\n", product.id);
    fclose(file);
    return;
}

printf("Enter Product Name: ");
scanf(" %[^\n]", product.name);
printf("Enter Product Price: ");
scanf("%f", &product.price);
printf("Enter Product Quantity: ");
scanf("%d", &product.quantity);

fprintf(file, "%d#//#%s#//#%.2f#//#%d\n", product.id, product.name, product.price, product.quantity);
fclose(file);
printf("Product added successfully.\n");
```

3. Add Sale Bill Functionality:

- **C Techniques:**
  - **File Handling (fopen, fprintf, fclose):** Saves bill details to the file.
  - **Structs:** Organizes bill details (bill number, date, user ID, product name, price).
  - **Mathematical Operations:** Calculates the total price.
  - **Conditional Statements (if-else):** Validates product availability and quantity.

- **Explanation:**
  This function generates a sale bill by calculating the total price based on the product ID and quantity. It ensures the product is available and saves the bill details to a file.

Code Snapshot:

```
FILE *file = fopen("sale_bills.txt", "a");
if (!file) {
    printf("Error opening sale bills file.\n");
    return;
}

Bill bill;
bill.bill_no = rand() % (9999 - 1000 + 1) + 1000;
time_t now = time(NULL);
strftime(bill.date_time, sizeof(bill.date_time), "%Y-%m-%d %H:%M:%S", localtime(&now));

printf("Enter Product ID: ");
scanf("%d", &bill.product.id);
printf("Enter Quantity: ");
scanf("%d", &bill.quantity);

// Calculate total price
bill.price = bill.product.price * bill.quantity;

fprintf(file, "%d#//#%s#//#%d#//#%s#//#%.2f\n",
        bill.bill_no, bill.date_time, bill.user.id, bill.product.name, bill.price);
fclose(file);
printf("Bill added successfully.\n");
```

4. Show Sale Bills Functionality:

- **C Techniques:**
  - **File Handling (fopen, fscanf, fclose):** Reads bill data from the file.
  - **Loops (while):** Iterates through all bill records.
  - **Formatting (printf):** Displays bill details in a structured manner.

- **Explanation:**
  This function retrieves and displays all sale bills stored in the file. It uses loops to read each record and formatting to present the data clearly.

Code Snapshot:

```
FILE *file = fopen("sale_bills.txt", "r");
if (!file) {
    printf("Error opening sale bills file.\n");
    return;
}

Bill bill;
while (fscanf(file, "%d#//#%[^#]#//#%d#//#%[^#]#//#%f\n",
              &bill.bill_no, bill.date_time, &bill.user.id, bill.product.name, &bill.price) != EOF) {
    printf("Bill No: %d, Date: %s, User ID: %d, Product: %s, Price: %.2f\n",
           bill.bill_no, bill.date_time, bill.user.id, bill.product.name, bill.price);
}

fclose(file);
```

5. Update Product Functionality:

- **C Techniques:**
  - **File Handling (fopen, fscanf, fprintf, fclose):** Reads and rewrites product data.
  - **Conditional Statements (if-else):** Locates the product and updates specific fields.
  - **Input Validation:** Ensures updated data is valid.

- **Explanation:**
  This function allows users to update product details such as name, price, or quantity. It ensures data accuracy by validating inputs and updating the file accordingly.

Code Snapshot:

```
FILE *file = fopen("products.txt", "r");
FILE *temp_file = fopen("temp.txt", "w");
if (!file || !temp_file) {
    printf("Error opening files.\n");
    return;
}

Product product;
int id_to_update, found = 0;
printf("Enter Product ID to update: ");
scanf("%d", &id_to_update);

while (fscanf(file, "%d#//#%[^#]#//#%f#//#%d\n",
              &product.id, product.name, &product.price, &product.quantity) != EOF) {
    if (product.id == id_to_update) {
        found = 1;
        printf("Enter New Price: ");
        scanf("%f", &product.price);
    }
    fprintf(temp_file, "%d#//#%s#//#%.2f#//#%d\n",
            product.id, product.name, product.price, product.quantity);
}

fclose(file);
fclose(temp_file);

if (found) {
    remove("products.txt");
    rename("temp.txt", "products.txt");
    printf("Product updated successfully.\n");
} else {
    remove("temp.txt");
    printf("Product not found.\n");
}
```

Complexity and Advanced Techniques:

- The system uses **file handling**, **structs**, **loops**, **conditional statements**, and **input validation** to ensure robust functionality.
- Advanced techniques like **random number generation** (for bill numbers) and **time manipulation** (for bill dates) add complexity and enhance the system's capabilities.

2.4 Output and User Interface

Inputs	Output
Login: User ID: 101 Password: password123	**Success:** "Login successful!" **Error:** "Invalid username or password."
Add User: User ID: 102 Name: John Doe Password: john123	**Success:** "User added successfully." **Error:** "User with ID 102 already exists."
Delete User: User ID: 102	**Success:** "User deleted successfully." **Error:** "User not found."
Add Product: Product ID: 501 Name: Laptop Price: 1200.50	**Success:** "Product added successfully." **Error:** "Product with ID 501 already exists."
Delete Product: Product ID: 501	**Success:** "Product deleted successfully." **Error:** "Product not found.
Add Sale Bill: Product ID: 501 Quantity: 2	**Success:** "Bill added successfully." **Error:** "Invalid product ID or quantity."
Show Sale Bills:	**Output: Displays a list of all sale bills with details (bill number, date, user ID, etc.).**

2.5 Summary

Future Expansion of the System:

1. **Inventory Management:**
    - **Function:** Add a feature to track low stock levels and automatically reorder products when inventory falls below a threshold.
    - **C Techniques:**
        - **File Handling:** To store and update inventory data.
        - **Conditional Statements:** To check stock levels and trigger reorder alerts.
        - **Loops:** To iterate through product records and identify low-stock items.
2. **Sales Reporting and Analytics:**
    - **Function:** Generate detailed sales reports, including daily, weekly, and monthly summaries, and visualize data using graphs.
    - **C Techniques:**
        - **File Handling:** To read sales data from files.
        - **Mathematical Operations:** To calculate totals, averages, and trends.
        - **External Libraries:** Use libraries like gnuplot (via system calls) to create graphs.
3. **Multi-User Support:**
    - **Function:** Allow multiple users to access the system simultaneously over a network.
    - **C Techniques:**
        - **Socket Programming:** To enable communication between clients and a central server.
        - **Threading:** To handle multiple user requests concurrently.
4. **Advanced Search and Filtering:**
    - **Function:** Add search and filtering options for users, products, and bills (e.g., search by date, category, or price range).
    - **C Techniques:**
        - **String Manipulation:** To implement search functionality.
        - **Sorting Algorithms:** To sort and filter data based on user criteria.
5. **Discounts and Promotions:**
    - **Function:** Apply discounts or promotions to specific products or bills.
    - **C Techniques:**

- **Mathematical Operations:** To calculate discounted prices.
- **Conditional Statements:** To apply discounts based on predefined rules.

6. **Data Backup and Recovery:**
   - **Function:** Implement automated data backup and recovery mechanisms to prevent data loss.
   - **C Techniques:**
     - **File Handling:** To create backup copies of data files.
     - **System Commands:** To schedule backups using external tools like cron (on Linux).
7. **User Activity Logging:**
   - **Function:** Log all user activities (e.g., login attempts, product updates, sales) for auditing purposes.
   - **C Techniques:**
     - **File Handling:** To store activity logs.
     - **Timestamps:** To record the date and time of each activity.
8. **Integration with Barcode Scanners:**
   - **Function:** Integrate barcode scanners to quickly input product IDs during billing.
   - **C Techniques:**
     - **External Device Handling:** Use libraries or system calls to interface with barcode scanners.
     - **String Manipulation:** To process scanned barcode data.
9. **Multi-Currency Support:**
   - **Function:** Allow the system to handle transactions in multiple currencies.
   - **C Techniques:**
     - **Mathematical Operations:** To convert prices between currencies.
     - **File Handling:** To store exchange rates.
10. **Customer Management:**
    - **Function:** Add a feature to manage customer details, track purchase history, and offer loyalty rewards.
    - **C Techniques:**
      - **Structs:** To organize customer data.
      - **File Handling:** To store and retrieve customer records.

Advanced C Techniques for Future Expansion:

- **Dynamic Memory Allocation (malloc, calloc, free):** To handle large datasets efficiently.
- **Data Structures (Linked Lists, Trees):** To manage complex relationships (e.g., product categories, customer hierarchies).
- **Database Integration:** Use libraries like SQLite to store and query data more efficiently.
- **Error Handling:** Implement robust error handling to ensure the system can recover from unexpected issues.

3 Appendixes

3.1 Meeting Report

Date	Activity	Picture of the Meeting(s)	Actions Taken	Next Action Plan

3.2 Full Coding

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <stdbool.h>
#include <windows.h>

// File Paths
#define USER_FILE "users.txt"
#define PRODUCT_FILE "products.txt"
#define BILL_FILE  "sale_bills.txt"
typedef struct {
    int id;
    char name[50];
    float price;
    int quantity_in_storage;
} Product;

typedef struct {
    int id;
    char name[50];
    char phone[50];
    int access; // binary access
    char password[50];
    float salary;
} User;

typedef struct {
    int id;
    char name[50];
    int quantity;
    float price;
    int sold_quantity;
} Products;

typedef struct {
    int bill_no;
    char date_time[50];
    User user;
    Product product;
    float price;
```

```
} Bill;

void Header(char Title[50]);
void Tabs(short tab);

int USER_ID ;

// Declaration of the manageInput function
void manageInput(void *variable, bool isInt, bool isFloat, bool isString);
void main_menu(User user);
bool check_Accessibility(int id);
void reports();
void show_access_denied();
void go_back();

//USER
void login();
void manage_users();
void read_users();
void add_user();
void delete_user();
void update_user();
int user_exists(int);

//PRODUCT
void manage_products();
void read_products(bool Bill);
void add_product();
void delete_product();
void update_product();

//bILLING
int check_bill_number_exists(int bill_number);
//int check_product_exists(const char* product_name);
int check_product_exists(const int product_id);
int search_sale_bill(int bill_number);
void billing_system();
void add_sale_bill();
void show_sale_bills();



int main() {
   login();

  // display_menu();
```

```
    return 0;
}

void manageInput(void *variable, bool isInt, bool isFloat, bool isString) {

      bool valid = false;

    while (!valid) {
        if (isInt) {
            int temp = 0;
            if (scanf("%d", &temp) == 1) {
                *(int *)variable = temp;
                valid = true;
            } else {
                printf("Invalid input. Please enter a valid integer.\n");
            }
            while (getchar() != '\n'); // Clear input buffer
        }

        if (isFloat && !valid) {
            float temp = 0.0;
            if (scanf("%f", &temp) == 1) {
                *(float *)variable = temp;
                valid = true;
            } else {
                printf("Invalid input. Please enter a valid float.\n");
            }
            while (getchar() != '\n'); // Clear input buffer
        }

        if (isString && !valid) {
            char *temp = (char *)variable;
            if (scanf("%49[^\n]", temp) == 1) { // Limit input to 49 characters
                valid = true;
            } else {
                printf("Invalid input. Please enter a valid string.\n");
            }
            while (getchar() != '\n'); // Clear input buffer
        }
    }

}


void Tabs(short tab){
```

```
    for(short i = 0 ; i< tab ; i++)
        printf("\t");

}

void Header(char title[50]){
    Tabs(4);
    printf("==================================================\n");//50
    Tabs(6);
    printf("%2s\n", title);
    Tabs(4);
    printf("==================================================\n");//50
}

bool check_Accessibility (int id){
    FILE* file = fopen(USER_FILE, "r");
    if (!file) {
        Tabs(4);
        system("\a");
        printf("Error opening users file.\n");
        return 0; // File does not exist, so the user can't exist
    }

    User user;

    while (fscanf(file, "%d#//#%[^#]#//#%[^#]#//#%d#//#%[^#]#//#%f\n",
                  &user.id, user.name, user.phone, &user.access, user.password,
&user.salary) != EOF) {
        if (user.id == id) {
                if(user.access == 1)
                {
                    fclose(file);
                    return true;
                }

        }
    }

    fclose(file);
    return false; // User does not exist
}

void display_menu() {
   system("cls");
    int choice;
```

```
    while (1) {
        Header("Main Menue");
        Tabs(4);
        printf("1. Manage Users\n");
          Tabs(4);
        printf("2. Manage Products\n");
          Tabs(4);
        printf("3. Billing System\n");
          Tabs(4);
        printf("4. Log-out\n");
          Tabs(4);
        printf("Enter your choice: ");
      //  scanf("%d", &choice);
      manageInput(&choice, true, false,false);

        switch (choice) {
            case 1:
                  Tabs(4);
                 printf("Loading...\n");
                 Sleep(100);
                 system("cls");
                manage_users();
                break;
            case 2:
                  Tabs(4);
                 printf("Loading...\n");
                 Sleep(100);
                 system("cls");
                manage_products();
                break;
            case 3:
                  Tabs(4);
                 printf("Loading...\n");
                 Sleep(100);
                 system("cls");
                billing_system();
                break;
            case 4:
                printf("\n");
                 Tabs(4);
                 printf("Logging out... Goodbye!\n");
                 Sleep(500);
                 system("cls");
                exit(0);//close thewindows
                break;
            default:
```

```
                Tabs(5);
                 system("\a");
               printf("Invalid choice. Please try again.\n");

        }
    }
}

#define TEMP_FILE "temp.txt"

void login() {
    User user;
    int id;
    char input_password[50];
    Header("LOG-IN SCREEN");
    Tabs(4);
    printf("Enter User ID: ");
    manageInput(&id,true,false,false);
    //scanf("%d", &id);
    Tabs(4);
    printf("- - - - - - - - - - - - - -\n");
    Tabs(4);
    printf("Enter Password: ");
    manageInput(input_password, false, false,true);
  // scanf("%s", input_password);

    bool valid = false;
    FILE* file = fopen(USER_FILE, "r"); //
    if (!file) {
        printf("\a");
        Tabs(4);
        printf("Error opening user file.\n");
        return;
    }

    while (fscanf(file, "%d#//#%[^#]#//#%[^#]#//#%d#//#%[^#]#//#%f\n",
                  &user.id, user.name, user.phone, &user.access, user.password,
&user.salary) != EOF) {
        if ((id == user.id)  && strcmp(input_password, user.password) == 0) {

            valid = true;
            USER_ID = user.id;
            //main_menu(user);
            break;
        }
    }
```

```
        fclose(file);
        if(valid == false)
        {
            Tabs(4);
            printf("Invalid username or password.\n");
        }
        else
             display_menu();
}



void manage_users() {

        if(!check_Accessibility(USER_ID)){
            system("cls");
            Header("Manage Users Screen");
            printf("YOU DON'T HAVE ACCESS TO MANAGE THE USERS...\n");
            system("pause");
            system("cls");
            return;

        }

    int choice;



    while (1) {
            system("cls");
            Header("Manage Users Screen");
        Tabs(4);
        printf("1. Show Users List\n");
         Tabs(4);
        printf("2. Add New User\n");
         Tabs(4);
        printf("3. Delete User\n");
         Tabs(4);
        printf("4. Update User\n");
         Tabs(4);
        printf("5. Go Back to Main Screen\n");
         Tabs(4);
        printf("Choose an option: ");
      //  scanf("%d", &choice);
       manageInput(&choice, true, false,false);
```

```
        switch (choice) {
        case 1:
             system("cls");
            read_users();
            break;
        case 2:
             system("cls");
            add_user();
            break;
        case 3:
             system("cls");
            delete_user();
            break;
        case 4:
             system("cls");
            update_user();
            break;
        case 5:
             Tabs(4);

            printf("Returning to the main menu...\n");
            Sleep(100);
            system("cls");
            return;
        default:
             system("\a");
             Tabs(4);
             printf("Invalid choice. Please try again.\n");
             Sleep(200);
             system("cls");
        }
    }
}

int user_exists(int  user_id) {
    FILE* file = fopen(USER_FILE, "r");
    if (!file) {
        Tabs(4);
        printf("\a");
        printf("Error opening users file.\n");
        return 0; // File does not exist, so the user can't exist
    }

    User user;
```

```
    // Read the file and search for the user ID
    while (fscanf(file, "%d#//#%[^#]#//#%[^#]#//#%d#//#%[^#]#//#%f\n",
                  &user.id, user.name, user.phone, &user.access, user.password,
&user.salary) != EOF) {
        if (user.id == user_id) {
            fclose(file);
            return 1; // User exists
        }
    }

    fclose(file);
    return 0; // User does not exist
}

void read_users() {

Header("User List Screen");
    FILE* file = fopen(USER_FILE, "r");
if (!file) {
        Tabs(4);
    printf("Could not open users file.\n");
    return;
}
printf("\n\n");
User user;
Tabs(2);
printf("+------ +--------------------+-----------------+--------+----------------
----+------------+\n");
Tabs(2);
printf("| ID   | Name               | Phone           | Access |
Password           | Salary     |\n");
Tabs(2);
printf("+------ +--------------------+-----------------+--------+----------------
----+------------+\n");


while (fscanf(file, "%d#//#%[^#]#//#%[^#]#//#%d#//#%[^#]#//#%f\n",
              &user.id, user.name, user.phone, &user.access, user.password,
&user.salary) != EOF) {
                  Tabs(2);
    // Print formatted user data with correct column widths
    printf("| %-5d | %-18s | %-15s | %-6d | %-18s | %-10.2f |\n",
           user.id, user.name, user.phone, user.access, user.password,
user.salary);
}
Tabs(2);
```

```
printf("+------ +--------------------+----------------+--------+----------------
----+------------+\n");

fclose(file);
Tabs(2);
system("pause");


}


//Add user access = 1 , Manage User = 2, Manage Product = 4, Billing System
access = 8, Report Access = 16
void add_user() {
    Header("Add A New User Screen");
    FILE* file = fopen(USER_FILE, "a");
    if (!file) {
            printf("\a");
            Tabs(4);
        printf("Error opening user file.\n");
        return;
    }

    User user;
    int temproray = 0;
    Tabs(4);
    printf("Enter User ID: ");
  //  scanf("%d", &user.id);
    manageInput( &temproray, true, false,false);
    user.id = (int)temproray;

    // Check if user ID already exists
    if (user_exists(user.id)) {
        printf("\a");
        Tabs(4);
        printf("User with ID %d already exists.\n", user.id);
        fclose(file);
        return;
}

Tabs(4);
    printf("Enter User Name: ");
    //scanf(" %[^\n]", user.name); //  read an entire line of input until a
newline (\n) is encountered
    manageInput(&user.name, false, false,true);
```

```
    Tabs(4);
    printf("Enter User Password: ");
   // scanf(" %[^\n]", user.password );
    manageInput(&user.password, false, false,true);
    Tabs(4);
    printf("Enter User Phone: ");
    //scanf(" %[^\n]", user.phone );
    manageInput(&user.phone, false, false,true);

    user.salary = 0;
    char chioce = 'n';
    user.access = 0;
    Tabs(4);
    printf ("Full Access ? [Y/N] ");
   // scanf(" %c", &chioce);
   manageInput(&chioce, false, false,true);

    if( chioce == 'Y' || chioce == 'y' ){
        user.access = 1;
    }

    else{
        user.access = 0;
        /*
        Tabs(4);
        printf( "Access for Main Menue Screen ? Y/N");
        manageInput(&chioce, false, false,true);

         if( chioce == 'Y' || chioce == 'y')
        {
            user.access += 1;
        }
Tabs(4);
        printf( "Access for Mange user ? Y/N ");
        manageInput(&chioce, false, false,true);

         if( chioce == 'Y' || chioce == 'y')
        {
            user.access += 2;
        }
Tabs(4);
        printf( "Access for Mange Product ? Y/N");
        manageInput(&chioce, false, false,true);

         if( chioce == 'Y' || chioce == 'y')
        {
```

```
            user.access += 4;
        }
Tabs(4);
        printf( "Access for Mange Billing System ? Y/N");
       manageInput(&chioce, false, false,true);

         if( chioce == 'Y' || chioce == 'y')
        {
            user.access += 8;
        }
Tabs(4);
        printf( "Access for Reports Screen ? Y/N");
        manageInput(&chioce, false, false,true);

         if( chioce == 'Y' || chioce == 'y')
        {
            user.access += 16;
        }
*/

    }

    printf("\n\n");
    Tabs(4);
    printf ("******New User Details******\n\n");
    Tabs(4);
    printf("User ID       : %d\n", user.id);
    Tabs(4);
    printf("User Name     : %s\n", user.name);
    Tabs(4);
    printf("User Phone    : %s\n", user.phone);
    Tabs(4);
    printf("User Access   : %d\n", user.access);
    Tabs(4);
    printf("User Password : %s\n", user.password);
    Tabs(4);
    printf("User Salary   : %f\n", user.salary);
    Tabs(4);
    printf("**************************\n");



    fprintf(file, "%d#//#%s#//#%s#//#%d#//#%s#//#%.2f\n",
        user.id, user.name, user.phone, user.access, user.password, user.salary);

    fclose(file);
```

```
    Tabs(4);
    printf("User added successfully.\n");
    Tabs(4);
    system("pause");

}

void delete_user() {
    Header("Delete User Screen");
    FILE* file = fopen(USER_FILE, "r");
    if (!file) {
            printf("\a");
            Tabs(4);
        printf("Could not open users file.\n");
        return;
    }

    FILE* temp_file = fopen(TEMP_FILE, "w");
    if (!temp_file) {
            printf("\a");
            Tabs(4);
        printf("Could not open temporary file.\n");
        fclose(file);
        return;
    }

    User user;
    int id_to_delete, found = 0;
    Tabs(4);
    printf("Enter User ID to delete: ");
   // scanf("%d", &id_to_delete);
    manageInput( &id_to_delete, true, false,false);

    while (fscanf(file, "%d#//#%[^#]#//#%[^#]#//#%d#//#%[^#]#//#%f\n",
        &user.id, user.name, user.phone, &user.access, user.password,
&user.salary) != EOF) { //EOF End of file
        if (user.id == id_to_delete) {
            found = 1;
            printf("\n");
            Tabs(4);
            printf("User Details:\n");
            Tabs(4);
            printf("ID       : %d\n", user.id);
             Tabs(4);
            printf("Name     : %s\n", user.name);
             Tabs(4);
```

```
            printf("Phone    : %s\n", user.phone);
             Tabs(4);
            printf("Access   : %d\n", user.access);
             Tabs(4);
            printf("Password : %c\n", user.password);
             Tabs(4);
            printf("Salary   : %.2f\n\n", user.salary);

            Tabs(4);

        }
        else {
            fprintf(temp_file, "%d#//#%s#//#%s#//#%d#//#%s#//#%.2f\n",
                user.id, user.name, user.phone, user.access, user.password,
user.salary);
        }
    }

    fclose(file);
    fclose(temp_file);

    if (found) {
        remove(USER_FILE);
        rename(TEMP_FILE, USER_FILE);
        printf("\n\n");
        Tabs(4);
        printf("User deleted successfully.\n");

    }
    else {
        printf("\n\n");
            Tabs(4);

        printf("User not found.\n");
        remove(TEMP_FILE);
    }
    Tabs(4);
    system("pause");
}

void update_user() {
    Header("Update User Screen");
    FILE* file = fopen(USER_FILE, "r");
    if (!file) {
        printf("\a");
        Tabs(4);
```

```
        printf("Could not open users file.\n");
        return;
    }

    FILE* temp_file = fopen(TEMP_FILE, "w");
    if (!temp_file) {
       printf("\a");
        Tabs(4);
        printf("Could not open temporary file.\n");
        fclose(file);
        return;
    }

    User user;
    int id_to_update, found = 0, update_choice;
    char choice ;
    Tabs(4);

    printf("Enter User ID to update: ");
    //scanf("%d", &id_to_update);
     manageInput( &id_to_update, true, false,false);


    while (fscanf(file, "%d#//#%[^#]#//#%[^#]#//#%d#//#%[^#]#//#%f\n",
        &user.id, user.name, user.phone, &user.access, user.password,
&user.salary) != EOF) {
        if (user.id == id_to_update) {
            found = 1;
            Tabs(4);
            printf("User Details:\n");
            Tabs(4);
            printf("ID       : %d\n", user.id);
             Tabs(4);
            printf("Name     : %s\n", user.name);
             Tabs(4);
            printf("Phone    : %s\n", user.phone);
             Tabs(4);
            printf("Access   : %d\n", user.access);
             Tabs(4);
            printf("Password : %c\n", user.password);
             Tabs(4);
            printf("Salary   : %.2f\n\n", user.salary);
            Tabs(4);


            printf ("Do you want to update User Name [Y/N]? ");
```

```
          // scanf(" %c", &choice);
           manageInput( &choice, false, false,true);

           if(choice == 'Y' || choice == 'y')
           {
               Tabs(4);
               printf ("Enter A new User Name : ");
              // scanf (" %c", &user.name);
                 manageInput( &user.name, false, false,true);
           }
           Tabs(4);
           printf ("Do you want to update User Password [Y/N]? ");
       //   scanf(" %c", &choice);
           manageInput( &choice, false, false,true);

           if(choice == 'Y' || choice == 'y')
           {
               Tabs(4);
               printf ("Enter A new User Password : ");
              // scanf (" %c", &user.password);
               manageInput( &user.password, false, false,true);
           }
           Tabs(4);
           printf ("Do you want to update User Phone [Y/N]? ");
       //   scanf(" %c", &choice);
            manageInput(& choice, false, false,true);
           if(choice == 'Y' || choice == 'y')
           {
               Tabs(4);
               printf ("Enter A new User Phone : ");
              // scanf ("%c", &user.phone)
               manageInput( &user.phone, true, false,false);
           }

           Tabs(4);
          printf("After Updating User's Details :\n");
           Tabs(4);
           printf("ID       : %d\n", user.id);
            Tabs(4);
           printf("Name     : %s\n", user.name);
            Tabs(4);
           printf("Phone    : %s\n", user.phone);
            Tabs(4);
           printf("Access   : %d\n", user.access);
            Tabs(4);
           printf("Password : %c\n", user.password);
```

```
            Tabs(4);
            printf("Salary   : %.2f\n", user.salary);

        }
        fprintf(temp_file, "%d#//#%s#//#%s#//#%d#//#%s#//#%.2f\n",
            user.id, user.name, user.phone, user.access, user.password,
user.salary);
    }

    fclose(file);
    fclose(temp_file);
 printf("\n");
    if (found) {
        remove(USER_FILE);
        rename(TEMP_FILE, USER_FILE);

        Tabs(4);
        printf("User updated successfully.\n");
    }
    else {
        printf("\a");
        Tabs(4);
        printf("User not found.\n");
        remove(TEMP_FILE);
    }
    Tabs(4);
    system("pause");
}



void manage_products() {

    system("cls");
    int choice;

    while (1) {
        Header("Manage Products Screen");
        Tabs(4);
        printf("1. Show Products List\n");
        Tabs(4);
        printf("2. Add New Product\n");
        Tabs(4);
        printf("3. Delete Product\n");
        Tabs(4);
```

```
        printf("4. Update Product\n");
        Tabs(4);
        printf("5. Go Back to Main Screen\n");
        Tabs(4);
        printf("Choose an option: ");
        //scanf("%d", &choice);
        manageInput( &choice, true, false,false);

        switch (choice) {
            case 1:
                system("cls");
                read_products(false);
                break;
            case 2:
                system("cls");
                add_product();
                break;
            case 3:
                system("cls");
                delete_product();
                break;
            case 4:
                system("cls");
                update_product();
                break;
            case 5:

                printf("\n");
                Tabs(4);
                printf("Returning to the main menu...\n");
                Sleep(100);
                 system("cls");
                return;
            default:

                Tabs(4);
                printf("Invalid choice. Please try again.\n");
                Sleep(200);
                system("cls");
        }
    }
}

void read_products(bool Bill) {
    Header("Show Porduct List Screen");
    FILE *file = fopen(PRODUCT_FILE, "r");
```

```
    if (!file) {
      printf("\a");
        Tabs(4);
        printf("Could not open products file.\n");
        return;
    }

    Products product;
    printf("\n");
     Tabs(3);
    printf("+------+-----------------------+-----------+----------------+\n");
     Tabs(3);
    printf("| ID   | Name                  | Price     | Sold Quantity  |\n");
     Tabs(3);
    printf("+------+-----------------------+-----------+----------------+\n");


    while (fscanf(file, "%d#//#%[^#]#//#%f#//#%d\n",
              &product.id, product.name, &product.price, &product.sold_quantity)
!= EOF) {


    product.name[strcspn(product.name, "\n")] = '\0';
       Tabs(3);

    printf("| %-4d | %-18s | %-10.2f | %-14d |\n",
           product.id, product.name, product.price, product.sold_quantity);
}
 Tabs(3);
 printf("+----+-----------------------+-----------+----------------+\n");
    fclose(file);
    if(Bill == false)
    {
            printf("\a");
            printf("\n");
            Tabs(4);
            system("pause");
            system("cls");
    }

}

void add_product() {
    Header("Add A New Product Screen");
    FILE *file = fopen(PRODUCT_FILE, "a");
    if (!file) {
```

```
       printf("\a");
        Tabs(4);
        printf("Could not open products file.\n");
        return;
    }

    Products product, temp;
    int product_exists = 0;
    Tabs(4);

    printf("Enter Product ID: ");
   // scanf("%d", &product.id);
    manageInput( &product.id, true, false,false);

    // Check if the product already exists
    while (fscanf(file, "%d#//#%[^#]#//#%f#//#%d\n",
                  &temp.id, temp.name, &temp.price, &temp.sold_quantity) != EOF)
{
        if (temp.id == product.id) {
            product_exists = 1;
            Tabs(4);
            printf("Product already exists:\n");
            Tabs(4);
            printf("ID      : %d\n", temp.id);
            Tabs(4);
            printf("Name    : %s\n", temp.name);
            Tabs(4);
            printf("Price   : %f\n", temp.price);
            Tabs(4);
            printf("Sold Qt : %d\n", temp.sold_quantity);
            break;
        }
    }

    if (product_exists) {
        Tabs(4);
        printf("This product is already added.\n");
        fclose(file);
        return;
    }
 Tabs(4);
    // Add new product
     printf("Enter Product Name: ");
     manageInput( &product.name, false, false,true);

    //scanf(" %[^\n]", product.name);
```

```
    //fgets(product.name);
    //product.name = getchar();

      // Clear the input buffer to handle the newline issue
   // while (getchar() != '\n');

   //  fgets( product.name, sizeof( product.name), stdin); // Reads a line of
input including spaces

    //scanf("%s", product.name);
    Tabs(4);
    printf("Enter Product Price: ");
  //  scanf("%f", &product.price);
    manageInput( &product.price, false, true,false);
    product.sold_quantity = 50; // Default quantity

    fprintf(file,  "%d#//#%-22s#//#%.2f#//#%d\n", product.id, product.name,
product.price, product.sold_quantity);
   //fprintf(file, "%d#//#%s#//#%.2f#//#%d\n", product.id, product.name,
product.price, product.sold_quantity);

    fclose(file);
     Tabs(4);
    printf("Product added successfully.\n");
            Tabs(4);
            printf("ID      : %d\n", product.id);
            Tabs(4);
            printf("Name    : %s\n", product.name);
            Tabs(4);
            printf("Price   : %f\n", product.price);
            Tabs(4);
            printf("Sold Qt : %d\n", product.sold_quantity);

    printf("\n");
    Tabs(4);
    system("pause");
    system("cls");
}

void delete_product() {
    Header("Delete Product Screen");
    FILE *file = fopen(PRODUCT_FILE, "r");
    if (!file) {
        printf("\a");
        Tabs(4);
        printf("Could not open products file.\n");
```

```
        return;
    }

    FILE *temp_file = fopen(TEMP_FILE, "w");
    if (!temp_file) {
        Tabs(4);
        printf("Could not open temporary file.\n");
        fclose(file);
        return;
    }

    Products product;
    int id_to_delete, found = 0;
 Tabs(4);
    printf("Enter Product ID to delete: ");
 //   scanf("%d", &id_to_delete);
     manageInput(&id_to_delete, true, false,false);

    while (fscanf(file, "%d#//#%[^#]#//#%f#//#%d\n",
                  &product.id, product.name, &product.price,
&product.sold_quantity) != EOF) {
        if (product.id == id_to_delete) {
            found = 1;
            printf("\n");
             Tabs(4);
            printf("Deleting Product:\n");

            Tabs(4);
             printf("ID      : %d\n", product.id);
              Tabs(4);
             printf("Name    : %s\n", product.name);
              Tabs(4);
             printf("Price   : %f\n", product.price);
              Tabs(4);
             printf("Sold Qt : %d\n", product.sold_quantity);
        } else {
            fprintf(temp_file, "%d#//#%s#//#%.2f#//#%d\n",
                    product.id, product.name, product.price,
product.sold_quantity);
        }
    }

    fclose(file);
    fclose(temp_file);
     printf("\n");
    if (found) {
```

```
        remove(PRODUCT_FILE);
        rename(TEMP_FILE, PRODUCT_FILE);
        Tabs(4);
        printf("Product deleted successfully.\n");
    } else {
        system("\a");
        Tabs(4);
        printf("Product not found.\n");
        remove(TEMP_FILE);
    }


    printf("\n");
    Tabs(4);
    system("pause");
    system("cls");
}

void update_product() {
    Header("Update Product Screen");
    FILE *file = fopen(PRODUCT_FILE, "r");
    if (!file) {
        printf("\a");
        Tabs(4);
        printf("Could not open products file.\n");
        return;
    }

    FILE *temp_file = fopen(TEMP_FILE, "w");
    if (!temp_file) {
        Tabs(4);
        printf("Could not open temporary file.\n");
        fclose(file);
        return;
    }

    Products product;
    int id_to_update, found = 0, update_choice;
    Tabs(4);
    printf("Enter Product ID to update: ");
    //scanf("%d", &id_to_update);
     manageInput(&id_to_update, true, false,false);
    while (fscanf(file, "%d#//#%[^#]#//#%f#//#%d\n",
                  &product.id, product.name, &product.price,
&product.sold_quantity) != EOF) {
        if (product.id == id_to_update) {
```

```
            found = 1;
            printf("\n");
            Tabs(4);
            printf("Updating Product:\n");
            Tabs(4);
            printf("ID       : %d\n", product.id);
            Tabs(4);
            printf("Name     : %s\n", product.name);
            Tabs(4);
            printf("Price    : %f\n", product.price);
            Tabs(4);
            printf("Sold Qt  : %d\n", product.sold_quantity);
            Tabs(4);
            printf("_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _\n");
            Tabs(4);
            printf("What do you want to update?\n");
            Tabs(4);
            printf("1. Price\n");
            Tabs(4);
            printf("2. Add Quantity to Storage\n");
            Tabs(4);
            printf("Choose an option: ");
           // scanf("%d", &update_choice);
            manageInput(&update_choice, true, false,false);

            if (update_choice == 1) {
                Tabs(4);
                printf("Enter New Price: ");
               // scanf("%f", &product.price);
                manageInput(&product.price, false, true,false);
            } else if (update_choice == 2) {
                int additional_quantity;
                Tabs(4);
                printf("Enter Quantity to Add: ");
                //scanf("%d", &additional_quantity);
                manageInput(&additional_quantity, true, false,false);
                product.sold_quantity += additional_quantity;
            } else {
                Tabs(4);
                printf("Invalid choice.\n");
            }
        }
        fprintf(temp_file, "%d#//#%s#//#%.2f#//#%d\n",
                product.id, product.name, product.price, product.sold_quantity);
    }
```

```
    fclose(file);
    fclose(temp_file);
    printf("\n");
    if (found) {
        remove(PRODUCT_FILE);
        rename(TEMP_FILE, PRODUCT_FILE);
        Tabs(4);
        printf("Product updated successfully.\n");
    } else {
        printf("\a");
        Tabs(4);
        printf("Product not found.\n");
        remove(TEMP_FILE);
    }

    printf("\n");
    Tabs(4);
    system("pause");
    system("cls");
}



int check_bill_number_exists(int bill_number) {

    FILE* file = fopen(BILL_FILE, "r");
    if (!file) {
        printf("\a");
        Tabs(4);
        printf("Error opening sale bills file.\n");
        return 0;
    }

    Bill bill;
    int found = 0;

    while (fscanf(file, "%d#//#%[^#]#//#%d#//#%[^#]#//#%f\n",
        &bill.bill_no, bill.date_time, &bill.user.id, bill.product.name,
&bill.price) != EOF) {
        if (bill.bill_no == bill_number) {
            found = 1;
            break;
        }
    }

    fclose(file);
```

```
    return found;
}

int check_product_exists(const int product_id) {
    FILE* file = fopen(PRODUCT_FILE, "r");
    if (!file) {
        printf("\a");
        Tabs(4);
        printf("Error opening product file.\n");
        return 0;
    }

    Product product;
    int found = 0;

    while (fscanf(file, "%d#//#%[^#]#//#%f#//#%d\n",
                  &product.id, product.name, &product.price,
&product.quantity_in_storage) != EOF) {
        if (product_id == product.id) {
            found = 1;

            break;
        }
    }

    fclose(file);
    return found;
}


int search_sale_bill(int bill_number) {
    FILE* file = fopen(BILL_FILE, "r");
    if (!file) {
        printf("\a");
        Tabs(4);
        printf("Error opening sale bills file.\n");
        return 0;
    }

    Bill bill;
    int found = 0;

    while (fscanf(file, "%d#//#%[^#]#//#%d#//#%[^#]#//#%f\n",
                  &bill.bill_no, bill.date_time, &bill.user.id,
bill.product.name, &bill.price) != EOF) {
        // Check if the current bill matches the desired bill number
```

```
        if (bill.bill_no == bill_number) {
                found = 1;
                Tabs(4);
                printf("Return Bill Details:\n");
                Tabs(4);
                printf("  Bill Number: %d\n", bill.bill_no);
                Tabs(4);
                printf("  Date & Time: %s\n", bill.date_time);
                Tabs(4);
                printf("  User ID: %d\n", bill.user.id);
                break;
                }
            }


    fclose(file);
    if(found == 1){
      FILE* file = fopen(BILL_FILE, "r");
        if (!file) {
            printf("\a");
            Tabs(4);
            printf("Error opening sale bills file.\n");
            return 0;
        }
        while (fscanf(file,"%d#//#%[^#]#//#%d#//#%[^#]#//#%f\n",
                  &bill.bill_no, bill.date_time, &bill.user.id,
bill.product.name, &bill.price) != EOF) {
        // Check if the current bill matches the desired bill number
        if (bill.bill_no == bill_number)
            {
                Tabs(4);
                printf("  ProductName: %s\n", bill.product.name );
                Tabs(4);
                printf("  Product ID : %d\n", bill.product.id);
                Tabs(4);
                printf("  Price      : %.2f\n", bill.product.price);

            }


        }
         fclose(file);
    }


    if (!found) {
        printf("\a");
```

```
        Tabs(4);
        printf("No bill found with the given number.\n");
        return 0;
    }
    return found;
}


void billing_system() {
    int choice;

    while (1) {
            Header("Billing System Screen");
            Tabs(4);
            printf("1. Add Sale Bill\n");
            Tabs(4);
            printf("2. Show Sale Bills\n");
            Tabs(4);
            printf("3. Go Back to Main Screen\n");
            Tabs(4);
            printf("Choose an option: ");

       manageInput(&choice, true, false,false);

        switch (choice) {
        case 1:
             system("cls");
             add_sale_bill();
            break;
        case 2:
             system("cls");
             show_sale_bills();
            break;
        case 3:
             Tabs(4);
             printf("Returning to the main menu...\n");
             Sleep(200);
             system("cls");
             display_menu();

            return;
        default:
            printf("\a");
            Tabs(4);
            printf("Invalid choice. Please try again.\n");
        }
```

```
    }
}

void add_sale_bill() {
    Bill bill;
    int found = 0;
    Header("Add A New Sale Bill Screen");


    do {
        bill.bill_no = rand() % (9999 - 1000 + 1) + 1000;
        found = check_bill_number_exists(bill.bill_no);
    } while (found);


    time_t now = time(NULL);
    strftime(bill.date_time, sizeof(bill.date_time), "%Y-%m-%d %H:%M:%S",
localtime(&now));
    Tabs(4);
    printf("Enter User ID: ");
    manageInput(&bill.user.id, true, false, false);

   read_products(true);


    float total_amount = 0.0f;
    Product product, product_for_sale;

    do {
            //read_products(true);
        printf("\n");
        Tabs(4);

        printf("Enter Product ID (or '0' to finish): ");
        manageInput(&product_for_sale.id, true, false, false);

        if (product_for_sale.id != 0) {

            found = check_product_exists(product_for_sale.id);

            if (found) {
                FILE* file = fopen(PRODUCT_FILE, "r");
                if (!file) {
                    system("\a");
                    Tabs(4);
                    printf("Error: Could not open products file.\n");
                    return;
```

```
                }

                while (fscanf(file, "%d#//#%[^#]#//#%f#//#%d\n",
                              &product.id, product.name, &product.price,
&product.quantity_in_storage) != EOF) {
                    if (product.id == product_for_sale.id) {
                             Tabs(4);
                        printf("Product Details:\n");
                        Tabs(4);
                        printf("  Name: %s\n", product.name);
                        Tabs(4);
                        printf("  Price: %.2f\n", product.price);
                        Tabs(4);
                        printf("Enter Quantity: ");
                        manageInput(&product.quantity_in_storage, true, false,
false);

                        if (product.quantity_in_storage <= 0 ||
product.quantity_in_storage > product.quantity_in_storage) {
                            printf("\a");
                            Tabs(4);
                            printf("Invalid quantity. Please enter a valid
amount.\n");
                            break;
                        }


                        float product_total = product.price *
product.quantity_in_storage;
                        total_amount += product_total;
                        Tabs(4);
                        printf("  Product Total: %.2f\n", product_total);

                        FILE* file2 = fopen("sale_bills.txt", "a");
                        if (!file2) {
                            system("\a");
                            Tabs(4);
                            printf("Error: Could not open sale bills file.\n");
                            fclose(file);
                            return;
                        }


                        fprintf(file2, "%d#//#%s#//#%d#//#%s#//#%.2f\n",
                                bill.bill_no, bill.date_time, bill.user.id,
product.name, product_total);
```

```
                        fclose(file2);
                        Tabs(4);
                        printf("Product added to the bill.\n");
                        break;
                    }
                }
                fclose(file);
            } else {
                printf("\a");
                Tabs(4);
                printf("Error: Product not found.\n");
                Sleep(100);
                return;
            }
        }
    } while (product_for_sale.id != 0);

    // Print bill summary
    Tabs(4);
    printf("\nBill Summary:\n");
    Tabs(4);
    printf("  Bill Number: %d\n", bill.bill_no);
    Tabs(4);
    printf("  Date & Time: %s\n", bill.date_time);
    Tabs(4);
    printf("  User ID: %d\n", bill.user.id);
    Tabs(4);
    printf("  Total Amount: %.2f\n", total_amount);
    Tabs(4);

    printf("Sale bill saved successfully.\n");

    printf("\n");
    Tabs(4);
    system("pause");
    system("cls");
}

void show_sale_bills() {
    Header("Show Sale Bills List Screen");
    FILE* file = fopen(BILL_FILE, "r");
    if (!file) {
        printf("\a");
        Tabs(4);
        printf("Error: Could not open sale bills file.\n");
```

```
        return;
    }

    Bill bill;

    Tabs(3);
    printf("+---------+----------------------+----------+----------------------
--+-----------+\n");
    Tabs(3);
    printf("| Bill No | Date & Time          | User ID  |
Product                | Price     |\n");
    Tabs(3);
    printf("+---------+----------------------+----------+----------------------
--+-----------+\n");


    while (fscanf(file, "%d#//#%[^#]#//#%d#//#%[^#]#//#%f\n",
            &bill.bill_no, bill.date_time, &bill.user.id, bill.product.name,
&bill.product.price) == 5)
            {
                Tabs(3);
                printf("| %5d   | %21s | %3d      | %12s  | %3.6f |\n",
                 bill.bill_no, bill.date_time, bill.user.id, bill.product.name,
bill.product.price);
        }
    Tabs(3);
    printf("+---------+--------------------+----------+-----------------------
+-----------+\n");

    fclose(file);
    printf("\n");
    Tabs(4);
    system("pause");
    system("cls");
}
```

UNIVERSITI MALAYSIA PERLIS
FACULTY OF ELECTRONIC ENGINEERING &TECHNOLOGY

NMJ11004 COMPUTER PROGRAMMING

Mini Project Assessment Rubrics

Technical Presentation/Demonstration/Viva (5%)

Group Name		Program Code	NMJ11004 Computer Programming
Group members and Matric No	1.	Group	
	2.	Date	
	3.	Lecturer	
	4.		
Project Title			

Criteria	Allocation Marks	Scale			Weight	Total Component Marks
		Excellent (4-5 marks)	Average (3-4 marks)	Poor (0-2 marks)		
Development Tool Usability (CO2, PO5)	5	Student shows ability to utilize development tool to create project, write and build C program correctly.	Student sometimes has problem to utilize development tool to create project, write and build C program.	Student unable to utilize development tool to create project, write and build C program.	1	

Program Execution (CO2, PO5)	10	Student shows ability to compile and execute the program using GNU compiler independently. This includes the ability to produce terminal windows, enter correct input and demonstrate intended result.	Student sometimes has problem to compile and execute program using GNU compiler without guidance. In other words, student is unable to enter correct input and/or demonstrate intended result.	Student is unable to compile and execute C program using GNU compiler. In other words, students is unable to produce terminal window and/or demonstrate the intended result.	2	
Error debugging (CO2, PO5)	15	Student is able to solve/debug all programming errors during the project development based on learned programming concept. Student also able to demonstrate program execution accurately without errors according to the intended project specification.	Student is able to somehow solve/debug most of the programming errors during the project development based on learned programming concept. However, there are some of the program features were not accurately executed according to the intended project specification.	Student unable to solve/debug most of the programming errors during the project development and unable to demonstrate a functional program according to the intended project specification.	3	
Technical Knowledge Mastery and Skill (CO2, PO5)	10	Project outcome demonstrated a level of excellent engineering mastery and skills marked by truly innovative, authentic, and provocative ideas, solutions, and designs to meet the project objective(s) using the development tools.	Project outcome demonstrated sufficient amount of technical knowledge and skill in the development of feasible solutions to meet the project objective using the development tools.	Project outcome demonstrated a lack of technical knowledge and skill in designing solutions to meet the project objective(s) using the development tools.	2	

Q&A (CO2, PO5)	10	Student demonstrates confidence, skill and proficiency in order to answer the question regarding the project elements such as development tools and design process. The answer given is accurate and extremely satisfactory.	Student somehow demonstrates confidence, skill and proficiency in order to answer the question regarding the project elements such as development tools and design process. Some of the answer given are correct and satisfactory.	Student demonstrates lack of confidence, skill and proficiency in order to answer the question regarding the project elements such as development tools and design process. The answer given is inaccurate and unsatisfactory.	2	
TOTAL	50			**TOTAL EVALUATED MARKS**		
				COMPONENT (CO2, PO5) ASSESSMENT MARKS = TOTAL EVALUATED MARKS X 0.05		

Project Rubric (Report) – 13%

Group Name:		Program Code:	NMJ110004 Computer Programming
Group Members and ID:	1.	Group:	
	2.	Date:	
	3.	Lecturer:	
Project Title:			

Assessment Criteria	**Marks Allocated**	**Marks (M)**						**Weighted Marks Obtained (WMO)**
		0	**1**	**2**	**3**	**4**	**5**	

(CO4, PO3) **Introduction** • Objective • Significant of the Project	10.00	Introduction, objective and purposed of the project is not clear at all. Project does not give any significant to user	Introduction, objective and purposed of the project is not clear. Only small can be understandable Project does give only small significant to user	Introduction, objective and purposed of the project is not clear. Only few can be understandable Project does give few significant to user	Introduction, objective and purposed of the project is clear but only some are understandable Project does give some significant to user	Introduction, objective and purposed of the project is moderate and some explanations are missing. Project does give a moderate significant to user	Introduction, objective and purposed of the project is excellent and clearly identify and explained. Project does give significant to user.	(=M X 2)
(CO4, PO3) **Project Flowchart or Pseudocode and User Interface** • Need to Identify correct input, process and output using appropriate C programming technique, explanation of the design and the flowchart of the proposed design.	10.00	No flowchart or pseudocode There is no identifying and analyzing of input, process and output has been shown. No explanation of C technique has been shown.	Flowchart or pseudocode is not clear and not correct Only little of input, process and output has been identifying and analyzing. No explanation of C technique has been shown.	Flowchart or pseudocode is clear but not correct Only some of input, process and output has been identifying and analyzing. Some explanation of C technique has been shown.	Flowchart or pseudocode is partially correct but not clear Only some of input, process and output has been identifying and analyzing. Show some explanation of C technique that has been used.	Flowchart or pseudocode is partially correct Good in identifying and analyzing the input, process, and output of the system. Good explanation of chosen C technique.	Flowchart or pseudocode is clear and correct Excellent and details of identifying and analyzing of input, process and output for each function of the system. Excellent explanation of chosen C technique.	(=M X 2)

(CO4, PO3) **C Code Contents** The program must be complete, and these project-required elements must be used in the code: • Structure • Functions • Pointers • Arrays • Control Flow/Loops • Repetition	20.00	Program is incomplete and does not fulfill the project requirements	Only 1 element used correctly	Only 2 other elements used correctly	Only 3 other elements used correctly	Only 4 other elements used correctly	5 and more other elements used correctly	(=*M X 2*)
(CO4, PO3) **Output and User Interface**	10.00	The program is does not produce any output and no test cases are used. No description of error checking Unable to identify any interface design.	The program is producing incorrect results, and incorrect test cases are used. Describe only 1 error checking Able to identify only one level of interface design.	The program is producing correct results but does not display correctly and incorrect test cases are used. Describe only 2 error checking. Able to identify few level of interface design.	The program produces correct results, display correctly but only uses partial correct test cases. Describe three error checking. Able to identify several level of interface design.	The program produces the correct results and displays them correctly and does use correct test cases. Describes several error checking. Able to identify all level of interface design.	The program meets all the specifications required and works. Describe all suitable error checking. Able to correctly identify all level of interface design.	(=*M X 2*)

(CO4, PO3) **Complexity of project** • Have clarity and visibility with where the coding is easier to backtrack. • Reusability - The code could be reused and flexible for future changes • Readability- easily readable and understandable	20.00	There is no complexity in working with the required attributes.	The complexity of the project involves only 1 attribute	The complexity of the project involves only 2 attributes	The complexity of the project involves 3 attributes	The complexity of the project involves 4 attributes	The complexity of the project involves MORE THAN required attributes	(=*M X 2*)

• Consistency and coupling between one code to another code								
(CO4, PO3) **C Coding Style** Good coding practices must be demonstrated in the code which includes: • Comments on important line of codes • Nicely written codes with consistent indentation • Organized and readable (code grouping) • Consistent naming scheme • Compact and effective codes	10.00	None of these elements are present in the code	Only 1 element present in the code	Only 2 elements present in the code	Only 3 elements present in the code	Only 4 elements present in the code	All elements present in the code	(=*M X2*)

(CO4, PO12) (3%) **Extendable of Programming Design** • the coding add new elements more beyond core knowledge • add on creativity	20.00	Makes NO references to previous learning and does not explain how the programmin g design can be expand in the future requirements .	Makes vague references to previous learning but does not apply knowledge and skills to demonstrate comprehension and performance in novel situations.	The completed codes add 1 new element beyond what was described in the task. Then shows evidence of applying that knowledge and those skills to demonstrate comprehension and performance in novel situations	The completed codes add 2 new elements beyond what was described in the task. Then shows evidence of applying that knowledge and those skills to demonstrate comprehension and performance in novel situations	The completed codes add 3 new elements beyond what was described in the task. Then shows evidence of applying that knowledge and those skills to demonstrate comprehension and performance in novel situations	The completed codes add 4 or more new elements beyond what was described in the task. Then shows evidence of applying that knowledge and those skills to demonstrate comprehension and performance in novel situations.	(=*M X2*)
Max. Marks	**100**					**Total Evaluated Marks (TEM)**		
						Total WMO.TPE (13%) i.e TEM x 0.13		

Peer Evaluation Assesment (2%)

(CO4, PO12)

Your Name:

Your Matrix No:

Your peer's name:

Your peer's matrix no:

	Good				Poor
Quality of Work Consider the degree to which your peer team members Provides work that is accurate and complete	5	4	3	2	1
Timeliness of Work Consider your peer member's timeliness of work	5	4	3	2	1
Task Support Consider the amount of task support the student Team member gives to others.	5	4	3	2	1
Interaction Consider how the peer members relates and communicates to others	5	4	3	2	1
Attendance Consider the student team member's attendance at the group meeting.	5	4	3	2	1
Responsibility Consider the ability of the student team member to carry out a chosen or assigned task, & reliability to complete tasks	5	4	3	2	1
Involvement Consider the extent to which the student team member participates in the exchange of information (does outside research, brings outside knowledge to group).	5	4	3	2	1
Leadership Consider how the team member engages in leadership activities.	5	4	3	2	1
Overall Performance Consider the overall performance of the student team member in the group.	5	4			